

ADAPTERS

A TYPE-THEORETIC FOUNDATION FOR TYPE CASTING

Formath Seminar – November 24th 2025

Meven LENNON-BERTRAND

Agda

LEMON

ROCQ



Idris

Have many form of **type casting**...

Agda

LEON

ROCQ



Idris

Have many form of **type casting**... and they're quite **complicated**.

Parametrized coercions #2455

[New issue](#)[Open](#)

coqbot opened on Dec 6, 2010

Member ...

Note: the issue was created automatically with bugzilla2github tool

Original bug ID: BZ#2455

From: **@robertkrebbbers**

Reported version: trunk

CC: **@Eelis**, **@JasonGross**



coqbot on Dec 6, 2010

Member Author ...

Comment author: **@robertkrebbbers**

Creating a parametrized coercion does not work. For example, I want to create a coercion from the positive elements of an arbitrary ordered ring into that ring.

(* Running Coq trunk r13689 *)

Section test.

(* Imagine R to be an ordered Ring *)

Context (R : Type).

(* Here we define the positive elements using a sigma type)

(In this example we are lazy and just take them all :) *)

Definition Pos := sig (fun x : R => True).

Assignees

No one assigned

Labels

part: coercions

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Code with agent mode

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Coercions again #403

New issue



Closed



leodemoura opened on Apr 14, 2021

Member ...

The Lean 4 coercions work much better than the Lean 3 ones, but they are still brittle and based on TC resolution. "Bad" instances often trigger non-termination. We can't support coercions from `A` to a subtype of `A` without allowing TC to invoke tactics, and we really don't want TC to invoke arbitrary tactics since it would make the system more complex, the caching mechanism will be less effective, and users will probably abuse the feature and create performance problems. Finally, the TC rules are to strict and prevent us from finding a coercion for

```
structure Foo (A : Sort _) := {foo : A}
structure Bar (A : Sort _) extends Foo A := {bar : A}
instance {A} : Coe (Bar A) (Foo A) := {coe := Bar.toFoo}
def getFoo {A} (F : Foo A) := F.foo
def bar : Bar Nat := {foo := 0, bar := 1}

#check getFoo bar -- fails because the expected type 'Foo ?A' contains a metavariable.
```



One option is to write an extensible coercion resolution procedure. Users would still be able to define (non-dependent) coercions using `instance` s, but the search and support for dependent coercions from `Prop` to `Bool` and `A` to subtype of `A` would be handwritten.



leodemoura added **refactoring** on Apr 14, 2021

Assignees

No one assigned

Labels

refactoring

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Code with agent mode

No branches or pull requests

Notifications

Customize

The `norm_cast` family of tactics.

A full description of the tactic, and the use of each theorem category, can be found at <https://arxiv.org/abs/2001.10594>.

```
def Lean.Elab.Tactic.NormCast.proveEqUsing
  (s : Meta.SimpTheorems) (a b : Expr) :
  MetaM (Option Meta.Simp.Result)
```

[source](#)

Proves `a = b` using the given simp set.

► Equations

```
def Lean.Elab.Tactic.NormCast.proveEqUsingDown
  (a b : Expr) :
  MetaM (Option Meta.Simp.Result)
```

[source](#)

Proves `a = b` by simplifying using move and squash lemmas.

► Equations

```
def Lean.Elab.Tactic.NormCast.mkCoe
  (e ty : Expr) :
  MetaM Expr
```

[source](#)

Constructs the expression `(e : ty)`.

► Equations

```
def Lean.Elab.Tactic.NormCast.isCoeOf?
  (e : Expr) :
  MetaM (Option Expr)
```

[source](#)

Checks whether an expression is the coercion of some other expression, and if so returns that expression.

► Equations

[return to top](#)

[source](#)

- Imports
- Imported by

Lean.Elab.Tactic.NormCast.proveEqUsing
Lean.Elab.Tactic.NormCast.proveEqUsingDown
Lean.Elab.Tactic.NormCast.mkCoe
Lean.Elab.Tactic.NormCast.isCoeOf?
Lean.Elab.Tactic.NormCast.isNumeral?
Lean.Elab.Tactic.NormCast.splittingProcedure
Lean.Elab.Tactic.NormCast.prove
Lean.Elab.Tactic.NormCast.upwardAndElim
Lean.Elab.Tactic.NormCast.numeralToCoe
Lean.Elab.Tactic.NormCast.
 elabNormCastConfig
Lean.Elab.Tactic.NormCast.derive
Lean.Elab.Tactic.NormCast.elabModCast
Lean.Elab.Tactic.NormCast.normCastTarget
Lean.Elab.Tactic.NormCast.normCastHyp
Lean.Elab.Tactic.NormCast.evalNormCast0
Lean.Elab.Tactic.NormCast.evalConvNormCast
Lean.Elab.Tactic.NormCast.evalPushCast
Lean.Elab.Tactic.NormCast.elabAddElim

Polarities: subtyping for datatypes #65

New issue



Closed



catalin-hritcu opened on Nov 29, 2014

Member



`(x:int{x>1} * y:int{y>1})` is not a subtype of `(int * int)`



catalin-hritcu added **kind/bug** on Nov 29, 2014

24 remaining items

Load more



nikswamy on Mar 14, 2022

Collaborator



Addressing this issue requires more research. Closing as a wontfix until then.



nikswamy closed this as completed on Mar 14, 2022

Assignees

No one assigned

Labels

component/language-design

component/metatheory

component/typechecker

hard

kind/enhancement

status/wont-fix

Relationships

None yet

Development

Code with agent mode



No branches or pull requests

Disable all subtyping by default? #4474

New issue 

 Closed



jespercockx opened on Feb 23, 2020

Member ...

In the light of historic and current issues involving subtyping (e.g. #1579 #2170 #2440 #3986 #4175 #4390 #4401) I am starting to wonder whether it is a good idea to have subtyping enabled by default in Agda. All dependent type theories with subtyping that are know either use coercive subtyping or restrict it to a very specific setting (i.e. cumulativity). On the other hand, Agda now has a notion of material subtyping that is used for several features: irrelevance, erasure, sized types, cumulativity, and cohesion. In particular, it seems that we do not yet fully understand how constraint solving and metavariables in such a setting are supposed to work.

In this light, I would like to discuss whether it is a good idea to have (material) subtyping enabled by default. Maybe it would be better to have a general flag `--no-subtyping` that disables material subtyping across the board? Things like irrelevance and erasure should still function with this option, though it might be necessary to eta-expand some functions by hand. Sized types and cumulativity would obviously not be compatible with this flag.

What do you think? Is this a good idea or do we need a less radical solution?



2

  jespercockx added `subtyping` `type: discussion` on Feb 23, 2020



nad on Feb 23, 2020

Contributor ...

I just [asked you](#) a similar question.

In particular, it seems that we do not yet fully understand how constraint solving and metavariables in such a setting are supposed to work.

Assignees

No one assigned

Labels

`subtyping` `type: discussion`

Type

No type

Projects

No projects

Milestone

 **2.6.1**
Closed on Mar 16, 2020, 100% complete

Relationships

None yet

Development

 Code with agent mode 

No branches or pull requests

Notifications [Customize](#)

Types can contain terms:
$$\frac{\Gamma \vdash A \quad \Gamma \vdash n : \mathbf{N}}{\Gamma \vdash \text{Vect } A \ n}$$

Types can contain terms:
$$\frac{\Gamma \vdash A \quad \Gamma \vdash n : \mathbf{N}}{\Gamma \vdash \text{Vect } A \ n}$$

A change with **many** consequences...

DEPENDENT TYPES IN ONE SLIDE

Types can contain terms:
$$\frac{\Gamma \vdash A \quad \Gamma \vdash n : \mathbf{N}}{\Gamma \vdash \text{Vect } A \ n}$$

A change with **many** consequences...

Type well-formation $\Gamma \vdash A$ Substitution during typing
$$\frac{\Gamma \vdash f : \Pi x:A.B \quad \Gamma \vdash u : A}{\Gamma \vdash fu : B[u/x]}$$

DEPENDENT TYPES IN ONE SLIDE

Types can contain terms:
$$\frac{\Gamma \vdash A \quad \Gamma \vdash n : \mathbf{N}}{\Gamma \vdash \text{Vect } A \ n}$$
 A change with **many** consequences...

Type well-formation $\Gamma \vdash A$ Substitution during typing
$$\frac{\Gamma \vdash f : \Pi x : A. B \quad \Gamma \vdash u : A}{\Gamma \vdash fu : B[u/x]}$$

Conversion/definitional equality
$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash A \equiv B}{\Gamma \vdash t : B}$$

Equivalence, congruent, and contains (at least) β rules
$$\frac{\Gamma, x : A \vdash t : B \quad \Gamma \vdash u : A}{\Gamma \vdash (\lambda x : A. t) u \equiv t[u/x] : B[u/x]}$$

DEPENDENT TYPES IN ONE SLIDE

Types can contain terms:
$$\frac{\Gamma \vdash A \quad \Gamma \vdash n : \mathbf{N}}{\Gamma \vdash \text{Vect } A \, n}$$

A change with **many** consequences...

Type well-formation $\Gamma \vdash A$ Substitution during typing
$$\frac{\Gamma \vdash f : \Pi x:A.B \quad \Gamma \vdash u : A}{\Gamma \vdash fu : B[u/x]}$$

Conversion/definitional equality
$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash A \equiv B}{\Gamma \vdash t : B}$$

Equivalence, congruent, and contains (at least) β rules
$$\frac{\Gamma, x:A \vdash t : B \quad \Gamma \vdash u : A}{\Gamma \vdash (\lambda x:A.t) u \equiv t[u/x] : B[u/x]}$$

Typing depends on the equational theory of the language!

WARMING-UP WITH SUBTYPING



Théo LAURENT



Kenji MAILLARD

Definitional Functoriality for Dependent (Sub)Types, ESOP 2025
(and Théo's PhD thesis)

What usersTM want:

$$\text{SUB} \frac{\Gamma \vdash_{\text{sub}} t : A \quad \Gamma \vdash_{\text{sub}} A \preccurlyeq A'}{\Gamma \vdash_{\text{sub}} t : A'}$$

What usersTM want:

$$\text{SUB} \frac{\Gamma \vdash_{\text{sub}} t : A \quad \Gamma \vdash_{\text{sub}} A \preccurlyeq A'}{\Gamma \vdash_{\text{sub}} t : A'}$$

Think of $\preccurlyeq$ as (set) inclusion.

$$\begin{array}{ccc} t & = & t \\ \cap & & \cap \\ \text{Tm}(A) & \subseteq & \text{Tm}(A') \end{array}$$

What usersTM want:

$$\text{SUB} \frac{\Gamma \vdash_{\text{sub}} t : A \quad \Gamma \vdash_{\text{sub}} A \preccurlyeq A'}{\Gamma \vdash_{\text{sub}} t : A'}$$

Think of $\preccurlyeq$ as (set) inclusion.

$$\begin{array}{ccc} t & = & t \\ \cap & & \cap \\ \text{Tm}(A) & \subseteq & \text{Tm}(A') \end{array}$$

Type theorists hate this:

- limited model: $A' \subseteq A \wedge B \subseteq B' \not\Rightarrow A \rightarrow B \subseteq A' \rightarrow B'$
- difficult meta-theory
- annoying to implement

What usersTM want:

$$\text{SUB} \frac{\Gamma \vdash_{\text{sub}} t : A \quad \Gamma \vdash_{\text{sub}} A \preccurlyeq A'}{\Gamma \vdash_{\text{sub}} t : A'}$$

Think of $\preccurlyeq$ as (set) inclusion.

$$\begin{array}{ccc} t & = & t \\ \cap & & \cap \\ \text{Tm}(A) \subseteq \text{Tm}(A') & & \end{array}$$

Type theorists hate this:

- limited model: $A' \subseteq A \wedge B \subseteq B' \not\Rightarrow A \rightarrow B \subseteq A' \rightarrow B'$
- difficult meta-theory
- annoying to implement

Someone in the room would say: **fibrations**! This is not how we'll cook it.

What type theorists want you to do:

$$\text{Coe} \frac{\Gamma \vdash_{\text{coe}} t : A \quad \Gamma \vdash_{\text{coe}} A \preccurlyeq A'}{\Gamma \vdash_{\text{coe}} \text{coe}_{A,A'} t : A'}$$

Explicit coe is much easier to model/study/implement!

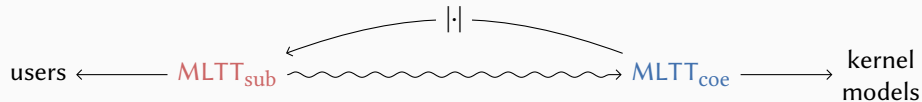
What type theorists want you to do:

$$\text{COE} \frac{\Gamma \vdash_{\text{coe}} t : A \quad \Gamma \vdash_{\text{coe}} A \preccurlyeq A'}{\Gamma \vdash_{\text{coe}} \text{coe}_{A,A'} t : A'}$$

Explicit coe is much easier to model/study/implement!

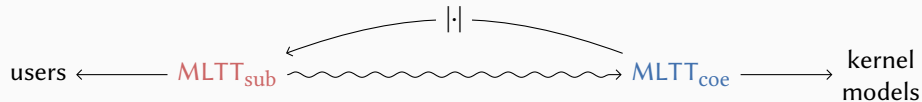
This is the work of a compiler!

LET'S COMPILE, THEN!



$$\text{SUB} \frac{\Gamma \vdash_{\text{sub}} t : A \quad \Gamma \vdash_{\text{sub}} A \preccurlyeq A'}{\Gamma \vdash_{\text{sub}} t : A'} \quad \rightsquigarrow \quad \text{COE} \frac{\tilde{\Gamma} \vdash_{\text{coe}} \tilde{t} : \tilde{A} \quad \tilde{\Gamma} \vdash_{\text{coe}} \tilde{A} \preccurlyeq \tilde{A}'}{\tilde{\Gamma} \vdash_{\text{coe}} \text{coe}_{\tilde{A}, \tilde{A}'} \tilde{t} : \tilde{A}'}$$

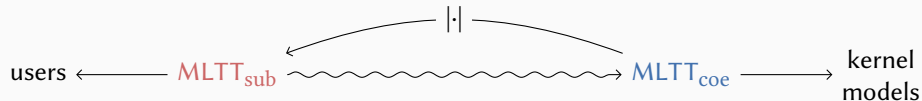
LET'S COMPILE, THEN!



$$\text{SUB} \frac{\Gamma \vdash_{\text{sub}} t : A \quad \Gamma \vdash_{\text{sub}} A \preccurlyeq A'}{\Gamma \vdash_{\text{sub}} t : A'} \quad \rightsquigarrow \quad \text{COE} \frac{\tilde{\Gamma} \vdash_{\text{coe}} \tilde{t} : \tilde{A} \quad \tilde{\Gamma} \vdash_{\text{coe}} \tilde{A} \preccurlyeq \tilde{A}'}{\tilde{\Gamma} \vdash_{\text{coe}} \text{coe}_{\tilde{A}, \tilde{A}'} \tilde{t} : \tilde{A}'}$$

Unable to unify "t" with "t".

LET'S COMPILE, THEN!



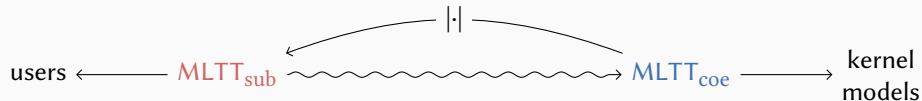
$$\text{SUB} \frac{\Gamma \vdash_{\text{sub}} t : A \quad \Gamma \vdash_{\text{sub}} A \preccurlyeq A'}{\Gamma \vdash_{\text{sub}} t : A'} \quad \rightsquigarrow \quad \text{COE} \frac{\tilde{\Gamma} \vdash_{\text{coe}} \tilde{t} : \tilde{A} \quad \tilde{\Gamma} \vdash_{\text{coe}} \tilde{A} \preccurlyeq \tilde{A}'}{\tilde{\Gamma} \vdash_{\text{coe}} \text{coe}_{\tilde{A}, \tilde{A}'} \tilde{t} : \tilde{A}'}$$

Unable to unify "t" with "t".

Set Printing Coercions.

Unable to unify "t'" with "t'".

LET'S COMPILE, THEN!



$$\text{SUB} \frac{\Gamma \vdash_{\text{sub}} t : A \quad \Gamma \vdash_{\text{sub}} A \preccurlyeq A'}{\Gamma \vdash_{\text{sub}} t : A'} \rightsquigarrow \text{COE} \frac{\tilde{\Gamma} \vdash_{\text{coe}} \tilde{t} : \tilde{A} \quad \tilde{\Gamma} \vdash_{\text{coe}} \tilde{A} \preccurlyeq \tilde{A}'}{\tilde{\Gamma} \vdash_{\text{coe}} \text{coe}_{\tilde{A}, \tilde{A}'} \tilde{t} : \tilde{A}'}$$

Unable to unify "t" with "t".

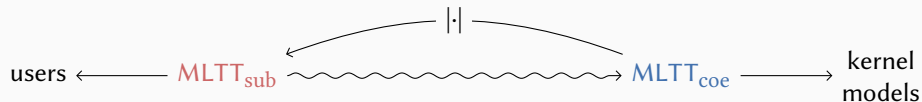
Set Printing Coercions.

Unable to unify "t'" with "t'".

Compilation should be **unambiguous**.

Necessary for compilation to **preserve typing**.

LET'S COMPILE, THEN!



$$\text{SUB} \frac{\Gamma \vdash_{\text{sub}} t : A \quad \Gamma \vdash_{\text{sub}} A \preccurlyeq A'}{\Gamma \vdash_{\text{sub}} t : A'} \rightsquigarrow \text{COE} \frac{\tilde{\Gamma} \vdash_{\text{coe}} \tilde{t} : \tilde{A} \quad \tilde{\Gamma} \vdash_{\text{coe}} \tilde{A} \preccurlyeq \tilde{A}'}{\tilde{\Gamma} \vdash_{\text{coe}} \text{coe}_{\tilde{A}, \tilde{A}'} \tilde{t} : \tilde{A}'}$$

Unable to unify "t" with "t".

Set Printing Coercions.

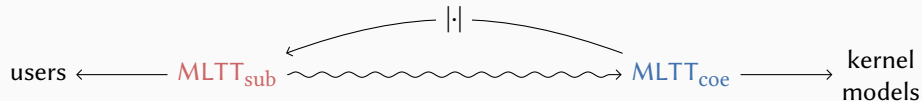
Unable to unify "t'" with "t'".

Compilation should be unambiguous.

Necessary for compilation to preserve typing.

Coherence: If $|t| = |u|$ then $t \equiv u$.

LET'S COMPILE, THEN!



$$\text{SUB} \frac{\Gamma \vdash_{\text{sub}} t : A \quad \Gamma \vdash_{\text{sub}} A \preccurlyeq A'}{\Gamma \vdash_{\text{sub}} t : A'} \rightsquigarrow \text{COE} \frac{\tilde{\Gamma} \vdash_{\text{coe}} \tilde{t} : \tilde{A} \quad \tilde{\Gamma} \vdash_{\text{coe}} \tilde{A} \preccurlyeq \tilde{A}'}{\tilde{\Gamma} \vdash_{\text{coe}} \text{coe}_{\tilde{A}, \tilde{A}'} \tilde{t} : \tilde{A}'}$$

Unable to unify "t" with "t".

Set Printing Coercions.

Unable to unify "t'" with "t'".

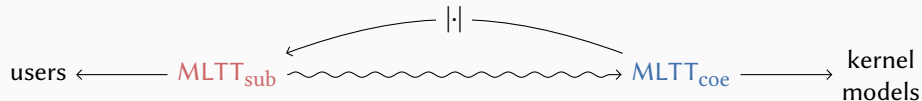
Compilation should be unambiguous.

Necessary for compilation to preserve typing.

Coherence: If $|t| = |u|$ then $t \equiv u$.

What equations do we need?

LET'S COMPILE, THEN!



$$\text{SUB} \frac{\Gamma \vdash_{\text{sub}} t : A \quad \Gamma \vdash_{\text{sub}} A \preccurlyeq A'}{\Gamma \vdash_{\text{sub}} t : A'} \rightsquigarrow \text{COE} \frac{\tilde{\Gamma} \vdash_{\text{coe}} \tilde{t} : \tilde{A} \quad \tilde{\Gamma} \vdash_{\text{coe}} \tilde{A} \preccurlyeq \tilde{A}'}{\tilde{\Gamma} \vdash_{\text{coe}} \text{coe}_{\tilde{A}, \tilde{A}'} \tilde{t} : \tilde{A}'}$$

Unable to unify "t" with "t".

Set Printing Coercions.

Unable to unify "t'" with "t'".

Compilation should be unambiguous.

Necessary for compilation to preserve typing.

Coherence: If $|t| = |u|$ then $t \equiv u$.

What equations do we need?

A **type-theoretic** question

NEW EQUATIONS FOR STRUCTURAL SUBTYPING

Focus = **structural** subtyping:

$$\frac{A' \preccurlyeq A \quad B \preccurlyeq B'}{A \rightarrow B \preccurlyeq A' \rightarrow B'}$$

$$\frac{A \preccurlyeq A'}{\text{List } A \preccurlyeq \text{List } A'}$$

$$\frac{A \preccurlyeq A' \quad B \preccurlyeq B'}{A \times B \preccurlyeq A' \times B'}$$

...

WHAT EQUATIONS DO WE NEED?

WHAT EQUATIONS DO WE NEED?

Computation equations:

$$\text{coe}_{\text{List } A, \text{List } A'} [] \equiv []$$

$$\text{coe}_{\text{List } A, \text{List } A'} (a :: l) \equiv (\text{coe}_{A, A'} a) :: \text{coe}_{\text{List } A, \text{List } A'} l$$

$$(\text{coe}_{A \rightarrow B, A' \rightarrow B'} f) u \equiv \text{coe}_{B, B'} (f (\text{coe}_{A', A} u))$$

$$\vdots$$

WHAT EQUATIONS DO WE NEED?

Computation equations:

$$\text{coe}_{\text{List } A, \text{List } A'} [] \equiv []$$

$$\text{coe}_{\text{List } A, \text{List } A'} (a :: l) \equiv (\text{coe}_{A, A'} a) :: \text{coe}_{\text{List } A, \text{List } A'} l$$

$$(\text{coe}_{A \rightarrow B, A' \rightarrow B'} f) u \equiv \text{coe}_{B, B'} (f (\text{coe}_{A', A} u))$$

$\vdots$

Functoriality equations:

$$\text{coe}_{A', A''} \text{coe}_{A, A'} l \equiv \text{coe}_{A, A''} l$$

$$\text{coe}_{A, A} l \equiv l$$

$\vdots$

$$\text{coe}_{\text{List } A, \text{List } A'} l := \text{map}_{\text{List}} \text{coe}_{A, A'} l \quad ?$$

$$\text{coe}_{\text{List } A, \text{List } A'} l := \text{map}_{\text{List}} \text{coe}_{A, A'} l \quad ?$$

Not in vanilla CIC/MLTT, where

$$\begin{aligned} \text{map}_{\text{List}} f (\text{map}_{\text{List}} g x) &\neq \text{map}_{\text{List}} (f \circ g) x \\ \text{map}_{\text{List}} \text{id } x &\neq x \end{aligned}$$

$$\text{coe}_{\text{List } A, \text{List } A'} l := \text{map}_{\text{List}} \text{coe}_{A, A'} l \quad ?$$

Not in vanilla CIC/MLTT, where

$$\begin{aligned} \text{map}_{\text{List}} f (\text{map}_{\text{List}} g x) &\not\equiv \text{map}_{\text{List}} (f \circ g) x \\ \text{map}_{\text{List}} \text{id } x &\not\equiv x \end{aligned}$$

Can we add these functoriality equations in?

?

K

ln?

Pierre Boutillier

PPS - Paris Diderot

pierre.boutillier@pps.univ-paris-diderot.fr

```
map : (a → b) → list a → list b
map f []      ~ []
map f (x :: xs) ~ f x :: map f xs
```

$$\begin{array}{l} (++) : \text{list } a \rightarrow \text{list } a \rightarrow \text{list } a \\ [] \quad ++ \text{ ys} \rightsquigarrow \text{ys} \\ x :: \text{xs} ++ \text{ys} \rightsquigarrow x :: (\text{xs} ++ \text{ys}) \end{array}$$

```
fold : (a → b → b) → b → list a → b
fold c n []      ~ n
fold c n (x :: xs) ~ c x (fold c n xs)
```

Table 1. $\delta\ell$ -rules - computational
$$\begin{array}{l} \Gamma \vdash f \equiv \lambda x. f \ x \quad : a \rightarrow b \\ \Gamma \vdash p \equiv (\pi_1 \ p, \ \pi_2 \ p) \quad : a * b \\ \Gamma \vdash u \equiv () \quad : 1 \end{array}$$
Table 2. η -rules - canonicity

`x` `a` `$\pi_1$` `$\pi_2$` `++ ys` `map f` `fold n c`

The second is the judgmental equality ($=$): it is typed, tractable

New

A Sound and

Guillaume Allais Con

University of Strath

{guillaume.allais, conor.mcbride}



Decidability of Conversion for Type Theory in Type Theory

ANDREAS ABEL, Gothenburg University, Sweden

JOAKIM ÖHMAN, IMDEA Software Institute, Spain

ANDREA VEZZOSI, Chalmers University of Technology, Sweden

Abstract

The definitional equality of an intensional type theory is a key component of type compatibility. Today's systems rely on operational semantics to compare expressions in types, but this approach is prone to type errors arising when evaluation fails to identify equal terms. If only the machine could decide definitional equality, we could propose a way to decide theories which supplement the ν -rules, rearranging the neutral parts of normal forms into a successful initial experiment.

We study a simple λ -calculus with primitive operations on lists and develop in Agda a decision procedure for an equational theory of λ -calculus with functor and fusion laws.

Keywords Normalization by Evaluation, Logical Relations, Typed Lambda Calculus, Map Fusion

1. Introduction

The programmer working in intensional type theory to 'obviously true' equations she wishes held by the program to typecheck without having to chase the semantics and brutally coerce them. In this article, we present a decision procedure for definitional equality, thus accommodating several types of fundamental relations. We distinguish three types of fundamental relations.

The first denotes computational rules: it is denoted by $\rightsquigarrow$ in its one step version or $\rightsquigarrow^*$ when its reflexive transitive closure is considered. In the few such rules which correspond to the equational theory, we write to define functions. They are referred to as ι (for pattern-matching on inductive data) and ϵ (for pattern-matching on inductive data) computationally just like the more common β -rule. The second is the judgmental equality ($=$).

Type theory should be able to handle its own meta-theory, both to justify its foundational claims and to obtain a verified implementation. At the core of a type checker for intensional type theory lies an algorithm to check equality of types, or in other words, to check whether two types are convertible. We have formalized in Agda a practical conversion checking algorithm for a dependent type theory with one universe à la Russell, natural numbers, and η -equality for Π types. We prove the algorithm correct via a Kripke logical relation parameterized by a suitable notion of equivalence of terms. We then instantiate the parameterized fundamental lemma twice: once to obtain canonicity and injectivity of type formers, and once again to prove the completeness of the algorithm. Our proof relies on inductive-recursive definitions, but not on the uniqueness of identity proofs. Thus, it is valid in variants of intensional Martin-Löf Type Theory as long as they support induction-recursion, for instance, Extensional, Observational, or Homotopy Type Theory.

CCS Concepts: • **Theory of computation** $\rightarrow$ **Type theory**; *Proof theory*;

Additional Key Words and Phrases: Dependent types, Logical relations, Formalization, Agda

ACM Reference Format:

Andreas Abel, Joakim Öhman, and Andrea Vezzosi. 2018. Decidability of Conversion for Type Theory in Type Theory. *Proc. ACM Program. Lang.* 2, POPL, Article 23 (January 2018), 29 pages. <https://doi.org/10.1145/3158111>

1 INTRODUCTION



Martin-Löf à la Coq

Arthur Adjedj
ENS Paris Saclay, Université
Paris-Saclay
Gif-sur-Yvette, France

Meven Lennon-Bertrand
University of Cambridge
Cambridge, United Kingdom

Kenji Maillard
Inria
Nantes, France

Pierre-Marie Pédro
Inria
Nantes, France

Loïc Pujet
University of Stockholm
Stockholm, Sweden

Abstract

We present an extensive mechanization of the metatheory of Martin-Löf Type Theory (MLTT) in the Coq proof assistant. Our development builds on pre-existing work in AGDA to show not only the decidability of conversion, but also the decidability of type checking, using an approach guided by bidirectional type checking. From our proof of decidability, we obtain a certified and executable type checker for a full-fledged version of MLTT with support for Π , Σ , $\mathbb{N}$, and Id types, and one universe. Our development does not rely on impredicativity, induction-recursion or any axiom beyond MLTT extended with indexed inductive types and a handful of predicative universes, thus narrowing the gap between the object theory and the metatheory to a mere difference in universes. Furthermore, our formalization choices are geared towards a modular development that relies on Coq's features, e.g. universe polymorphism and metaprogramming with tactics.

Keywords: Dependent type system, Bidirectional typing, Log-

checker is spent on establishing meta-theoretic properties, which are necessary to ensure termination of the type checker but have little to do with its concrete implementation.

Acknowledging this tension leads to two radically different approaches. On the one hand, one can simply postulate normalization, to better concentrate on the difficulties faced when certifying a realistic type-checker. The most ambitious project to date that follows this approach is META-Coq [Sozeau, Anand, et al. 2020; Sozeau, Forster, et al. 2023], which formalizes a nearly complete fragment of Coq's type system and provides a certified type checker aiming for execution in a realistic context, after extraction. On the other hand, one can concentrate on normalization and decidability of conversion, which are the most difficult theoretical problems. The most advanced formalizations on that end are Abel, Öhman, et al. [2017] and Wieczorek and Biernacki [2018]. The first, in AGDA, shows decidability of conversion, but does not provide an executable conversion checker. The second, in Coq, certifies a conversion checker designed for execution after extraction, but supports a type theory that is

Type Theory

al claims and to obtain
an algorithm to check
ve formalized in Agda
se à la Russell, natural
relation parameterized
damental lemma twice:
e completeness of the
ess of identity proofs.
rt induction-recursion,

Agda

Type Theory in Type
doi.org/10.1145/3158111

THEOREMS!

MLTT_{coe} is a nice type theory

We can design MLTT_{coe} with all these equations and good type theoretic properties:

- logical consistency
- decidability of conversion and type-checking

Featuring List () Π , Σ , $\mathbb{W}$, $+$, $=\dots$ ()

THEOREMS!

MLTT_{coe} is a nice type theory

We can design MLTT_{coe} with all these equations and good type theoretic properties:

- logical consistency
- decidability of conversion and type-checking

Featuring List () Π , Σ , W , $+$, $= \dots$ ()

And it is a good target

There is an elaboration $\text{MLTT}_{\text{sub}} \rightsquigarrow \text{MLTT}_{\text{coe}}$ which preserves conversion and typing. Moreover, it is an inverse of erasure, and elaboration is thus coherent.

“To compile structural implicit subtyping,
you need **exactly** functoriality equations.”

“To compile structural implicit subtyping,
you need **exactly** functoriality equations.”

But: missing a **general framework**.

STRUCTURAL CASTS AND FUNCTORIAL TYPES



Arthur ADJEDJ



Thibaut BENJAMIN



Kenji MAILLARD

AdapTT: Functoriality for Dependent Type Casts, POPL 2026

(STRUCTURAL) CASTS EVERYWHERE

Explicit subtyping:

cast along subtyping derivations.

$$\frac{\Gamma \vdash A' \preccurlyeq A \quad \Gamma \vdash B \preccurlyeq B'}{\Gamma \vdash A \rightarrow B \preccurlyeq A' \rightarrow B'} \quad \Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a' : A'$$
$$\Gamma \vdash (\text{coe}_{A \rightarrow B, A' \rightarrow B'} f) a' \equiv \text{coe}_{B, B'}(f \text{ coe}_{A', A} a) : B'$$

(STRUCTURAL) CASTS EVERYWHERE

Explicit subtyping:

cast along subtyping derivations.

$$\frac{\frac{\Gamma \vdash A' \preccurlyeq A \quad \Gamma \vdash B \preccurlyeq B'}{\Gamma \vdash A \rightarrow B \preccurlyeq A' \rightarrow B'} \quad \Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a' : A'}{\Gamma \vdash (\text{coe}_{A \rightarrow B, A' \rightarrow B'} f) a' \equiv \text{coe}_{B, B'}(f \text{ coe}_{A', A} a) : B'}$$

Observational equality:

cast along equality proofs.

$$\frac{\frac{\Gamma \vdash e_A : A' = A \quad \Gamma \vdash e_B : B = B'}{\Gamma \vdash e' := \dots : A \rightarrow B = A' \rightarrow B'} \quad \Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a' : A'}{\Gamma \vdash \text{trans}_{A \rightarrow B, A' \rightarrow B'}(e', f) a' \equiv \text{trans}_{B, B'}(e_B, f \text{ trans}_{A', A}(e_A, a')) : B'}$$

Dynamic typing[10, 11]:

cast always allowed (might fail).

$$\frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a' : A'}{\Gamma \vdash (\langle A' \rightarrow B' \Leftarrow A \rightarrow B \rangle f) a' \equiv \langle B' \Leftarrow B \rangle (f \langle A \Leftarrow A' \rangle a') : B'}$$

(STRUCTURAL) CASTS EVERYWHERE

Explicit subtyping:

cast along subtyping derivations.

$$\frac{\frac{\Gamma \vdash A' \preccurlyeq A \quad \Gamma \vdash B \preccurlyeq B'}{\Gamma \vdash A \rightarrow B \preccurlyeq A' \rightarrow B'} \quad \Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a' : A'}{\Gamma \vdash (\text{coe}_{A \rightarrow B, A' \rightarrow B'} f) a' \equiv \text{coe}_{B, B'}(f \text{ coe}_{A', A} a) : B'}$$

Observational equality:

cast along equality proofs.

$$\frac{\frac{\Gamma \vdash e_A : A' = A \quad \Gamma \vdash e_B : B = B'}{\Gamma \vdash e' := \dots : A \rightarrow B = A' \rightarrow B'} \quad \Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a' : A'}{\Gamma \vdash \text{trans}_{A \rightarrow B, A' \rightarrow B'}(e', f) a' \equiv \text{trans}_{B, B'}(e_B, f \text{ trans}_{A', A}(e_A, a')) : B'}$$

Dynamic typing[10, 11]:

cast always allowed (might fail).

$$\frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a' : A'}{\Gamma \vdash (\langle A' \rightarrow B' \Leftarrow A \rightarrow B \rangle f) a' \equiv \langle B' \Leftarrow B \rangle (f \langle A \Leftarrow A' \rangle a') : B'}$$

There's a general pattern!

(STRUCTURAL) CASTS EVERYWHERE

Explicit subtyping:

cast along subtyping derivations.

$$\frac{\frac{\Gamma \vdash A' \preccurlyeq A \quad \Gamma \vdash B \preccurlyeq B'}{\Gamma \vdash A \rightarrow B \preccurlyeq A' \rightarrow B'} \quad \Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a' : A'}{\Gamma \vdash (\text{coe}_{A \rightarrow B, A' \rightarrow B'} f) a' \equiv \text{coe}_{B, B'}(f \text{ coe}_{A', A} a) : B'}$$

Observational equality:

cast along equality proofs.

$$\frac{\frac{\Gamma \vdash e_A : A' = A \quad \Gamma \vdash e_B : B = B'}{\Gamma \vdash e' := \dots : A \rightarrow B = A' \rightarrow B'} \quad \Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a' : A'}{\Gamma \vdash \text{trans}_{A \rightarrow B, A' \rightarrow B'}(e', f) a' \equiv \text{trans}_{B, B'}(e_B, f \text{ trans}_{A', A}(e_A, a')) : B'}$$

Dynamic typing[10, 11]:

cast always allowed (might fail).

$$\frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a' : A'}{\Gamma \vdash (\langle A' \rightarrow B' \leftarrow A \rightarrow B \rangle f) a' \equiv \langle B' \leftarrow B \rangle (f \langle A \leftarrow A' \rangle a') : B'}$$

There's a general pattern! **Functoriality?**

Functor = a mapping between two categories:

- acts on objects and **arrows**
- preserves identities and composition

Functor = a mapping between two categories:

- acts on objects and arrows
- preserves identities and composition

1. Make types into a category
2. Describe the source of type formers
3. Show functoriality for our favourite type formers
4. Profit!

Functor = a mapping between two categories:

- acts on objects and arrows
- preserves identities and composition

1. **Make types into a category**
2. Describe the source of type formers
3. Show functoriality for our favourite type formers
4. Profit!

THE CATEGORY OF TYPES AND ADAPTERS

Adapters = “the data along which you can cast”

$$\frac{a : A \Rightarrow A' \quad t : A}{t\langle a \rangle : A'}$$

$$\frac{t : A}{t\langle \text{id}_A \rangle \equiv t : A}$$

$$\frac{a : A \Rightarrow A' \quad a' : A' \Rightarrow A'' \quad t : A}{t\langle a' \circ a \rangle \equiv t\langle a \rangle \langle a' \rangle : A''}$$

THE CATEGORY OF TYPES AND ADAPTERS

Adapters = “the data along which you can cast”

$$\frac{a : A \Rightarrow A' \quad t : A}{t\langle a \rangle : A'}$$

$$\frac{t : A}{t\langle \text{id}_A \rangle \equiv t : A}$$

$$\frac{a : A \Rightarrow A' \quad a' : A' \Rightarrow A'' \quad t : A}{t\langle a' \circ a \rangle \equiv t\langle a \rangle \langle a' \rangle : A''}$$

A **family** of type theories:

- **Subtyping**: $A \Rightarrow B$ corresponds to $A \preccurlyeq B$. **Uniqueness!**

THE CATEGORY OF TYPES AND ADAPTERS

Adapters = “the data along which you can cast”

$$\frac{a : A \Rightarrow A' \quad t : A}{t\langle a \rangle : A'}$$

$$\frac{t : A}{t\langle \text{id}_A \rangle \equiv t : A}$$

$$\frac{a : A \Rightarrow A' \quad a' : A' \Rightarrow A'' \quad t : A}{t\langle a' \circ a \rangle \equiv t\langle a \rangle \langle a' \rangle : A''}$$

A family of type theories:

- Subtyping: $A \Rightarrow B$ corresponds to $A \preceq B$. Uniqueness!
- **Full function space**: given any $f : A \rightarrow B$ we get $\underline{f} : A \Rightarrow B$ (and $t\langle f \rangle \equiv f \, t$)

THE CATEGORY OF TYPES AND ADAPTERS

Adapters = “the data along which you can cast”

$$\frac{a : A \Rightarrow A' \quad t : A}{t\langle a \rangle : A'}$$

$$\frac{t : A}{t\langle \text{id}_A \rangle \equiv t : A}$$

$$\frac{a : A \Rightarrow A' \quad a' : A' \Rightarrow A'' \quad t : A}{t\langle a' \circ a \rangle \equiv t\langle a \rangle \langle a' \rangle : A''}$$

A family of type theories:

- Subtyping: $A \Rightarrow B$ corresponds to $A \preceq B$. Uniqueness!
- Full function space: given any $f : A \rightarrow B$ we get $\underline{f} : A \Rightarrow B$ (and $t\langle f \rangle \equiv f \, t$)
- **Observational equality**: given $e : A = B$ we get $\underline{e} : A \Rightarrow B$
- **Dynamic typing**: $A \Rightarrow B$ always inhabited

THE CATEGORY OF TYPES AND ADAPTERS

Adapters = “the data along which you can cast”

$$\frac{a : A \Rightarrow A' \quad t : A}{t\langle a \rangle : A'}$$

$$\frac{t : A}{t\langle \text{id}_A \rangle \equiv t : A}$$

$$\frac{a : A \Rightarrow A' \quad a' : A' \Rightarrow A'' \quad t : A}{t\langle a' \circ a \rangle \equiv t\langle a \rangle\langle a' \rangle : A''}$$

A family of type theories:

- Subtyping: $A \Rightarrow B$ corresponds to $A \preceq B$. Uniqueness!
 - Full function space: given any $f : A \rightarrow B$ we get $\underline{f} : A \Rightarrow B$ (and $t\langle f \rangle \equiv f \, t$)
 - Observational equality: given $e : A = B$ we get $\underline{e} : A \Rightarrow B$
 - Dynamic typing: $A \Rightarrow B$ always inhabited
- } Need non-uniqueness too

THE CATEGORY OF TYPES AND ADAPTERS

Adapters = “the data along which you can cast”

$$\frac{a : A \Rightarrow A' \quad t : A}{t\langle a \rangle : A'}$$

$$\frac{t : A}{t\langle \text{id}_A \rangle \equiv t : A}$$

$$\frac{a : A \Rightarrow A' \quad a' : A' \Rightarrow A'' \quad t : A}{t\langle a' \circ a \rangle \equiv t\langle a \rangle\langle a' \rangle : A''}$$

A family of type theories:

- Subtyping: $A \Rightarrow B$ corresponds to $A \leq B$. Uniqueness!
 - Full function space: given any $f : A \rightarrow B$ we get $\underline{f} : A \Rightarrow B$ (and $t\langle f \rangle \equiv f \, t$)
 - Observational equality: given $e : A = B$ we get $\underline{e} : A \Rightarrow B$
 - Dynamic typing: $A \Rightarrow B$ always inhabited
- } Need non-uniqueness too

Abstractly:

- a category $\mathbf{Ty}_\Gamma$ of types and adapters
- $\mathbf{Tm}_\Gamma : \mathbf{Ty}_\Gamma \rightarrow \mathbf{Set}$ is a functor

(CwF-style reinvention of comprehension categories, see also Coraglia, Najmaei.)

Functor = a mapping between two categories:

- acts on objects and arrows
- preserves identities and composition

1. Make types into a category
2. **Describe the source of type formers**
3. Show functoriality for our favourite type formers
4. Profit!

WHAT IS IN A TYPE FORMER

Inductive List (X : Type) : Type := ...

Inductive W (X : Type) (Y : X → Type) : Type := ...

A type former is specified by a **context**

WHAT IS IN A TYPE FORMER

Inductive List (X : Type) : Type := ...

Inductive W (X : Type) (Y : X → Type) : Type := ...

A type former is specified by a **context**, with

- type variables: $\Gamma_{\text{List}} := X:\text{Ty}$ $\Gamma_{\rightarrow} := (X:\text{Ty}), (Y:\text{Ty})$

WHAT IS IN A TYPE FORMER

Inductive List (X : Type) : Type := ...

Inductive W (X : Type) (Y : X → Type) : Type := ...

A type former is specified by a **context**, with

- type variables: $\Gamma_{\text{List}} := X : \text{Ty}$ $\Gamma_{\rightarrow} := (X : \text{Ty}), (Y : \text{Ty})$
- *dependent* type variables: $\Gamma_{\text{W}} := (X : \text{Ty}), (Y : X. \text{Ty})$

WHAT IS IN A TYPE FORMER

Inductive List (X : Type) : Type := ...

Inductive W (X : Type) (Y : X → Type) : Type := ...

A type former is specified by a **context**, with

- type variables: $\Gamma_{\text{List}} := X : \text{Ty}$ $\Gamma_{\rightarrow} := (X : \text{Ty}), (Y : \text{Ty})$
- *dependent* type variables: $\Gamma_{\text{W}} := (X : \text{Ty}), (Y : X \rightarrow \text{Ty})$
- term variables: $\Gamma_{=} := (X : \text{Ty}), (x : X), (y : X)$

WHAT IS IN A TYPE FORMER

Inductive List (X : Type) : Type := ...

Inductive W (X : Type) (Y : X → Type) : Type := ...

A type former is specified by a **context**, with

- type variables: $\Gamma_{\text{List}} := X : \text{Ty}$ $\Gamma_{\rightarrow} := (X : \text{Ty}), (Y : \text{Ty})$
- *dependent* type variables: $\Gamma_{\text{W}} := (X : \text{Ty}), (Y : X. \text{Ty})$
- term variables: $\Gamma_{=} := (X : \text{Ty}), (x : X), (y : X)$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \rightarrow B}$$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash (A, B) : \Gamma_{\rightarrow}}$$

But wait! We have a way to relate two substitutions $\Gamma \vdash \sigma, \tau : \Gamma \rightarrow$:

Adapters!

But wait! We have a way to relate two substitutions $\Gamma \vdash \sigma, \tau : \Gamma_{\rightarrow}$:

Adapters!

We need **variance** information, though:

$$\Gamma_{\rightarrow} := (X: \text{Ty}_{-})(Y: \text{Ty}_{+}) \quad \Rightarrow \quad \frac{\Gamma \vdash a : A' \Rightarrow A \quad \Gamma \vdash b : B \Rightarrow B'}{\Gamma \vdash a \rightarrow b : A \rightarrow B \Rightarrow A' \rightarrow B'}$$

But wait! We have a way to relate two substitutions $\Gamma \vdash \sigma, \tau : \Gamma_{\rightarrow}$:

Adapters!

We need **variance** information, though:

$$\Gamma_{\rightarrow} := (X: \text{Ty}_{-})(Y: \text{Ty}_{+}) \quad \Rightarrow \quad \frac{\Gamma \vdash a : A' \Rightarrow A \quad \Gamma \vdash b : B \Rightarrow B'}{\Gamma \vdash (a, b) : (A, B) \Rightarrow_{\Gamma_{\rightarrow}} (A', B')}$$

But wait! We have a way to relate two substitutions $\Gamma \vdash \sigma, \tau : \Gamma_{\rightarrow}$: *Adapters!*

We need **variance** information, though:

$$\Gamma_{\rightarrow} := (X: \text{Ty}_{-})(Y: \text{Ty}_{+}) \quad \Rightarrow \quad \frac{\Gamma \vdash a : A' \Rightarrow A \quad \Gamma \vdash b : B \Rightarrow B'}{\Gamma \vdash (a, b) : (A, B) \Rightarrow_{\Gamma_{\rightarrow}} (A', B')}$$

A very general rule:

$$\frac{\Gamma \vdash A \quad \Delta \vdash \sigma, \tau : \Gamma \quad \Delta \vdash \mu : \sigma \Rightarrow_{\Gamma} \tau}{\Delta \vdash A[\mu] : A[\sigma] \Rightarrow A[\tau]}$$

All types are functors

But wait! We have a way to relate two substitutions $\Gamma \vdash \sigma, \tau : \Gamma_{\rightarrow}$: *Adapters!*

We need **variance** information, though:

$$\Gamma_{\rightarrow} := (X: \text{Ty}_{-})(Y: \text{Ty}_{+}) \quad \Rightarrow \quad \frac{\Gamma \vdash a : A' \Rightarrow A \quad \Gamma \vdash b : B \Rightarrow B'}{\Gamma \vdash (a, b) : (A, B) \Rightarrow_{\Gamma_{\rightarrow}} (A', B')}$$

A very general rule:

$$\frac{\Gamma \vdash A \quad \Delta \vdash \sigma, \tau : \Gamma \quad \Delta \vdash \mu : \sigma \Rightarrow_{\Gamma} \tau}{\Delta \vdash A[\mu] : A[\sigma] \Rightarrow A[\tau]}$$

All types are functors, obtained compositionally from functoriality of each type former.

- A 2-category **Ctx** of contexts, substitutions and transformations
- A 2-functor $\text{Ty} : \mathbf{Ctx} \rightarrow \mathbf{Cat}$ (maps Γ to the category $\mathbf{Ty}_\Gamma$, $\llbracket \cdot \rrbracket$ is the action on 2-arrows)
- A “dependent 2-functor” $\text{Tm} : (\Gamma : \mathbf{Ctx}) \rightarrow (\text{Ty}(\Gamma) \rightarrow \mathbf{Set})$;
- Local representability (type and term variables);
- a 2-functor $\cdot^- : \mathbf{Ctx}^{\text{co}} \rightarrow \mathbf{Ctx}$ to interpret negative variance.

Lots of data and equations to unpack!

Functor = a mapping between two categories:

- acts on objects and arrows
- preserves identities and composition

1. Make types into a category
2. Describe the source of type formers
3. **Show functoriality for our favourite type formers**
4. Profit!

Type specification recipe

1. Type formation rule $A \rightarrow B$
2. Constructor (λ) and eliminator (app)
3. Computation rule for each constructor-destructor combination (β)
4. Extensionality rule (η) (optional)

Type specification recipe, **adapted**

1. **Give the type former's context** $(\Gamma \rightarrow)$, and **derive**
 - 1.1 the type formation rule $A \rightarrow B$
 - 1.2 **the adapter formation rule** $a \rightarrow b$
2. Constructor (λ) and eliminator (app)
3. Computation rule for each constructor-destructor combination (β)
4. Extensionality rule (η) (optional)
5. **Computation rule for the adapter**

$$(f\langle a \rightarrow b \rangle) u \equiv (f\ u\langle a \rangle)\langle b \rangle$$

$$\Gamma_{\Pi} := (X: \tau_{y_-}), (Y: X. \tau_{y_+})$$

$$\Gamma_{\Sigma} := (X: \tau_{y_+}), (Y: X. \tau_{y_+})$$

$$\Gamma_{\Pi} := (X: \mathsf{Ty}_-, (Y: X. \mathsf{Ty}_+))$$

$$\Gamma_{\Sigma} := (X: \mathsf{Ty}_+, (Y: X. \mathsf{Ty}_+))$$

$$\frac{\Delta \vdash a : A' \Rightarrow A \quad \Delta, x: A' \vdash b : B[x\langle a \rangle/x] \Rightarrow B'}{\Delta \vdash \Pi a.b : \Pi(x: A).B \Rightarrow \Pi(x: A').B'}$$

$$\frac{\Delta \vdash a : A \Rightarrow A' \quad \Delta, x: A \vdash b : B \Rightarrow B'[x\langle a \rangle/x]}{\Delta \vdash \Sigma a.b : \Sigma(x: A).B \Rightarrow \Sigma(x: A').B'}$$

OUR FAVOURITE TYPE FORMERS

$$\Gamma_{\Pi} := (X: \mathsf{Ty}_{-}), (Y: X. \mathsf{Ty}_{+})$$

$$\Gamma_{\Sigma} := (X: \mathsf{Ty}_{+}), (Y: X. \mathsf{Ty}_{+})$$

$$\frac{\Delta \vdash a : A' \Rightarrow A \quad \Delta, x: A' \vdash b : B[x\langle a \rangle/x] \Rightarrow B'}{\Delta \vdash \Pi a.b : \Pi(x: A).B \Rightarrow \Pi(x: A').B'}$$

$$\frac{\Delta \vdash a : A \Rightarrow A' \quad \Delta, x: A \vdash b : B \Rightarrow B'[x\langle a \rangle/x]}{\Delta \vdash \Sigma a.b : \Sigma(x: A).B \Rightarrow \Sigma(x: A').B'}$$

$$\Gamma_{\Pi} := (X: \mathsf{Ty}_-), (Y: X. \mathsf{Ty}_+)$$

$$\Gamma_{\Sigma} := (X: \mathsf{Ty}_+), (Y: X. \mathsf{Ty}_+)$$

$$\frac{\Delta \vdash a : A' \Rightarrow A \quad \Delta, x: A' \vdash b : B[x\langle a \rangle/x] \Rightarrow B'}{\Delta \vdash \Pi a.b : \Pi(x: A).B \Rightarrow \Pi(x: A').B'} \quad \frac{\Delta \vdash a : A \Rightarrow A' \quad \Delta, x: A \vdash b : B \Rightarrow B'[x\langle a \rangle/x]}{\Delta \vdash \Sigma a.b : \Sigma(x: A).B \Rightarrow \Sigma(x: A').B'}$$

$$\frac{\Delta \vdash a : A' \Rightarrow A \quad \Delta, (x: A') \vdash b : B[x\langle a \rangle/x] \Rightarrow B' \quad \Delta \vdash f : \Pi(x: A).B \quad \Delta \vdash u : A'}{\Delta \vdash f\langle \Pi a.b \rangle u \equiv (f \textcolor{red}{u}\langle \textcolor{red}{a} \rangle)\langle b[u/x] \rangle : B'[u/x]}$$

$$\frac{\Delta \vdash a : A \Rightarrow A' \quad \Delta, (x: A) \vdash b : B \Rightarrow B'[x\langle a \rangle/x] \quad \Delta \vdash p : \Sigma(x: A).B}{\Delta \vdash \pi_1 (p\langle \Sigma a.b \rangle) \equiv (\pi_1 p)\langle a \rangle : A' \quad \Delta \vdash \pi_2 (p\langle \Sigma a.b \rangle) \equiv (\pi_2 p)\langle b[\pi_1 p/x] \rangle : B'[(\pi_1 p)\langle a \rangle/x]}$$

OUR FAVOURITE TYPE FORMERS

$$\Gamma_{\Pi} := (X: \text{Ty}_-), (Y: X. \text{Ty}_+)$$

$$\Gamma_{\Sigma} := (X: \text{Ty}_+), (Y: X. \text{Ty}_+)$$

$$\frac{\Delta \vdash a : A' \Rightarrow A \quad \Delta, x: A' \vdash b : B[x\langle a \rangle/x] \Rightarrow B'}{\Delta \vdash \Pi a.b : \Pi(x: A).B \Rightarrow \Pi(x: A').B'} \quad \frac{\Delta \vdash a : A \Rightarrow A' \quad \Delta, x: A \vdash b : B \Rightarrow B'[x\langle a \rangle/x]}{\Delta \vdash \Sigma a.b : \Sigma(x: A).B \Rightarrow \Sigma(x: A').B'}$$

$$\frac{\Delta \vdash a : A' \Rightarrow A \quad \Delta, (x: A') \vdash b : B[x\langle a \rangle/x] \Rightarrow B' \quad \Delta \vdash f : \Pi(x: A).B \quad \Delta \vdash u : A'}{\Delta \vdash f\langle \Pi a.b \rangle u \equiv (f \text{ } u\langle a \rangle)\langle b[u/x] \rangle : B'[u/x]}$$

$$\frac{\Delta \vdash a : A \Rightarrow A' \quad \Delta, (x: A) \vdash b : B \Rightarrow B'[x\langle a \rangle/x] \quad \Delta \vdash p : \Sigma(x: A).B}{\Delta \vdash \pi_1(p\langle \Sigma a.b \rangle) \equiv (\pi_1 p)\langle a \rangle : A' \quad \Delta \vdash \pi_2(p\langle \Sigma a.b \rangle) \equiv (\pi_2 p)\langle b[\pi_1 p/x] \rangle : B'[(\pi_1 p)\langle a \rangle/x]}$$

A bit intense... but **clear guidelines**.

Functor = a mapping between two categories:

- acts on objects and arrows
- preserves identities and composition

1. Make types into a category
2. Describe the source of type formers
3. Show functoriality for our favourite type formers
4. **Profit!**

```
Inductive list (A : Type) : Type :=  
  | nil : list A  
  | cons (a : A) (l : list A) : A  
  
Inductive W (A : Type) (B : A -> Type) : Type :=  
  | sup (a : A) (rec : B a -> W A B) : W A B  
  
Inductive eq (A : Type) : A -> A -> Type :=  
  | eq_refl (a : A) : eq A a a
```



```
Inductive list (A : Type+) : Type+ :=  
  | nil : list A  
  | cons (a : A) (l : list A) : A  
  
Inductive W (A : Type+) (B : A -> Type-) : Type :=  
  | sup (a : A) (rec : B a -> W A B) : W A B  
  
Inductive eq (A : Type) : A -> A -> Type :=  
  | eq_refl (a : A) : eq A a a
```

Idea:

1. Include **variance** in the types' context
2. Derive the adapter's type from the context
3. **Derive the adapter's computation rule from the constructors' description**

```
Inductive list (A : Type+) : Type+ :=  
  | nil : list A  
  | cons (a : A) (l : list A) : A  
  
Inductive W (A : Type+) (B : A -> Type-) : Type :=  
  | sup (a : A) (rec : B a -> W A B) : W A B  
  
Inductive eq (A : Type) : A -> A -> Type :=  
  | eq_refl (a : A) : eq A a a
```

Idea:

1. Include variance in the types' context
2. Derive the adapter's type from the context
3. Derive the adapter's computation rule from the constructors' description

All the hard work is done!

WRAPPING UP

- Meta-theory of AdapTT
- Alternative presentation of type variables?
- Experimental implementation
- Explore instances of the framework (cumulativity, subset types, records...)
- More category theory (Clean presentation of dependent variables?)

Structural casts $\leftrightarrow$ Functorial type formers

Structural casts $\leftrightarrow$ Functorial type formers

We can design **better coercions...**

Structural casts $\leftrightarrow$ Functorial type formers

We can design **better coercions...**

... but we have to push conversion **beyond mere computation!**

Structural casts $\leftrightarrow$ Functorial type formers

We can design **better coercions**...

... but we have to push conversion **beyond mere computation!**

THANKS!